

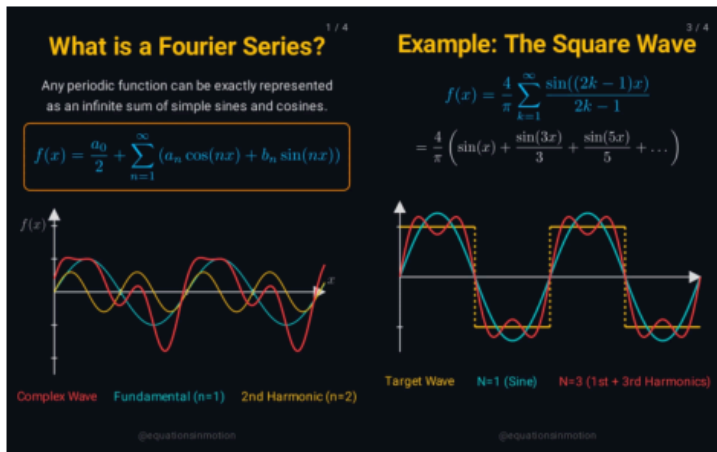
FINALS GRADING RUBRIC

General grading note

- Students answer any 10 questions, but are free to answer more, even all 15! Each question is worth 3 points. **Total score is capped at 30** regardless of how many extra questions a student answers and what total score she/he gets.
- Use Point adjustment in gradescope to give partial credit. provide appropriate comments for deduction.
- Overall, let's be quite lenient! Rather than look for exact wording that matches the rubrics, please read through to look for the right answer, even fragments of them - and award points accordingly.

Q1 — 3 points

Q1. How does Fourier decomposition (and its counterpart - Fourier synthesis) relate to a neural network? We WENT OVER this in class. Hint: overall signal, littles, big, patterns in data.



A. A neural network functions as a universal approximator, where the overall network learns to approximate the patterns in the input (training) data. In that sense it's analogous to the Fourier transform where a complex (periodic) signal is approximated by a set of $\sin()$ and $\cos()$ functions which together reproduce the signal. In our analogy, neurons are equivalent to the $\sin()$ and $\cos()$ units.

+1 Identifies the "big = sum of littles" idea (signal as sum of basis functions).

+2 Maps synthesis (weighted sum of basis fns / harmonics) to NN forward pass / inference (or mentions universal approximation).

Q2 — 3 points

Q2. When we do BI, we talk about a conceptual 'microscope' to zoom in and out of data.

What specifically in the data model (ie schema) enables this (makes it possible to zoom in/out)?

Provide an example of this (a use case where the analyst zooms in and out).

A. Dimension tables.

+1 Names dimension hierarchies / dimension tables as the enabler.

+2 Provides a concrete drill-down/roll-up example along at least one hierarchy (time, geography, product, etc.).

Q3 — 3 points

Q3. We very briefly covered 'GPSETC' (6 items, one for each letter) that are peripheral to the core database concepts, noting they are important nonetheless.

Pick any three of the six, and briefly explain how they relate to the 'new' AI (GenAI, agentic AI, LLMs/FMs...). In other words, why is 'GPSETC' suddenly more urgent+important, related to how 'AI' is starting to shape our world in sudden, unexpected ways (not all good)?

A.

+1 Picks 3 distinct items from GPSETC and identifies what each stands for.

+1 Connects each chosen item to a real GenAI/agentic-AI concern (training-data leaks, deepfakes, opacity, regulation, autonomous action, etc.)

+1 Conveys why the issue is more urgent now — speed/scale/autonomy of modern AI vs. the pre-LLM world.

Q4 — 3 points

Q4.

Choropleths are a specific form of data viz, where a single quantity is plotted on a spatial map (region, city, country...).

What are three examples of CLASSED choropleths that we would want to display? For example, median income could be classed and displayed for each state on the US map (or each province in China :-)).

A.

+1 *3 for each example (each example must be distinct valid quantity)

(examples: Unemployment rate by US county, COVID-19 case rate (cases per 100K) by, country, Median household income by US ZIP code)

Examples must be quantitative. No points for categorical answers.

Q5 — 3 points

Q5. "Many problems in CS can be effectively solved by introducing a level of indirection!"

How does the above aphorism/wisecrack/meme/... apply, to the following items?
You don't have to write a lot but you do need to be precise.

A. front-end dataflow graphs (eg Pig, GraphLINQ...) running on backend parallel processing platforms (Hadoop, Naiad...)

B. pub-sub (publisher/subscriber, ie producer/consumer) systems

C. agents (in agentic AI) and their tool-calling

A.

+1 (A) Identifies **Musketeer** - the dataflow DAG / compiler as the indirection between frontend graphs and the parallel backend (decoupling, optimizer, swappable backend).

+1 (B) Identifies **Kafka or Kinesis** as the indirection that decouples publishers from subscribers (in space/time/sync).

+1 (C) Identifies the tool interface / function schema / **MCP**/CLI scripts as the indirection between the LLM and real systems (swap, sandbox, no retraining).

Q6 — 3 points

Q6.

Here is some data, specified via 'minified' [unformatted] JSON:

```
{"errors":[{"location":"query","param":"open","value":["false","false"],"msg":"Invalid value"}]}
```

A. format it so it's better readable by humans, ie. use whitespace and newline (like you would with code!).

B. express it (the data) in words, eg. 'errors' is a key... Be sure to use 'array', 'object' etc. in your description ☺ It should be possible to read your description and derive (write down) from it alone, the complete JSON shown above.

A.

+1 Part A: produces correct, well-indented, syntactically valid pretty-printed JSON (whitespace + newlines).

```
{
  "errors": [
    {
      "location": "query",
      "param": "open",
      "value": [
        "false",
        "false"
      ],
      "msg": "Invalid value"
    }
  ]
}
```

+1 Part B: correctly identifies the outer structure as an object containing an array (uses both terms).

+1 Part B: lists all four inner keys with their correct value types — including the nested array of two strings [correctly identifies the outer structure as an object containing an array (uses both terms); lists all four inner keys with their correct value types — including the nested array of two strings].

For part B, ok to say **‘the overall JSON object has a single key called errors which has an array value, which consists of just one element, an object; that object has three keys (location, param, value)+values; the third key called ‘value’ has an array value with two elements’**. The wording doesn't need to be exactly like what is stated here, but needs to capture the nestedness (object, array, object, array).

Q7 — 3 points

Q7. ‘AI slop’ in AI-generated code is increasingly being exposed (by brave whistleblower coders, and disgruntled CEOs who are souring on over-hyped promises etc). In other words, AI doesn't magically write ‘good’ code - instead it merely creates ‘code that runs’ (software architecture practices be damned, lol).

The screenshot shows an article titled "Why AI Coding Tools Still Fail in Production". The text discusses the challenges of AI-generated code, noting that while AI agents can generate code, the costs of running them can be prohibitive. It mentions that some companies are cutting off free trials and limiting access to their coding models. The article also notes that AI-generated code is often "slop" and that human intervention is still needed. A workflow diagram is shown, illustrating the process of generating code, reviewing it, and then refining it through a series of steps: (1) Generate Code, (2) Human Review Layer, (3) Fixing + Comments, (4) Refactor + Fix, and (5) Merge or Reject. The diagram highlights that only step (1) is done by AI, while the other steps are done by humans.

In the box on the right above, of the 6 steps, only one ('AI Generates Code') is by AI, the other 5 are by humans - we need to spend much effort to keep AI from screwing up! Question: WHY is this happening (what is the origin of the errors in AI-created code)? To answer, think 'deterministic' (human coder) vs 'non-deterministic' (AI).

A. We are replacing well-designed and reasoned code, with output from a probabilistic token generator =, ie. an LLM (even though agentic wrappers around it might involve reasoning components, it's still not the kind of rigorous reasoning that is possible via symbolic AI).

The human steps are needed, to constrain the AI into producing reasonable code - by providing structured instruction, review and possible refactoring. In other words the AI's role is restricted to just be a code generator, with the assumption that the code might need to be modified before being accepted for deployment.

We can be rather lean about the rubrics - the answer simply needs to contain a reasonable explanation.

- +1 Names the deterministic/intentional nature of human coding (reasons about spec, types, invariants).
- +1 Names the non-deterministic / probabilistic / token-sampling nature of LLM code generation.
- +1 Connects that gap to specific failure modes (hallucination, plausibility ≠ correctness, no semantic model, no verification) — explaining why the 5 human steps are needed.

Q8 — 3 points

Q8. USC's DPS publishes an annual safety report, with tables that look like so:

Crime Report Statistics for the Annual Security and Fire Safety Report 2025 | UPC

Offense	2024 Residence Hall				2024 Total On Campus				2023 Residence Hall				2023 Total On Campus			
	2024 Public Property	2024 Non-Campus	2024 Public Property	2024 Non-Campus	2023 Public Property	2023 Non-Campus	2023 Public Property	2023 Non-Campus	2023 Public Property	2023 Non-Campus	2023 Public Property	2023 Non-Campus	2023 Public Property	2023 Non-Campus	2023 Public Property	2023 Non-Campus
Criminal Offenses																
Murder/Non-Negligent Manslaughter	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
Manslaughter by Negligence	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Rape	7	9	0	5	12	16	1	5	16	26	3	6	16	26	3	6
Fondling	5	16	4	4	2	17	7	1	2	12	10	6	2	12	10	6
Incest	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Statutory Rape	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0
Robbery	4	13	4	3	0	9	19	2	1	11	22	2	1	11	22	2
Aggravated Assault	6	16	8	1	1	11	15	7	3	13	28	8	3	13	28	8
Burglary	8	19	0	24	11	32	0	20	6	22	0	25	6	22	0	25
Motor Vehicle Theft	0	278	3	33	0	246	21	30	3	268	17	34	3	268	17	34
Arson	3	7	0	1	4	6	0	1	1	2	0	1	1	2	0	1

CA Penal Code defines electric/motorized scooters/bicycles/skateboards as a vehicle for the purposes of motor vehicle theft. Of the 314 motor vehicle thefts reported in 2024, 287 are electric scooters, bikes, skateboards, or golf carts. Of the 297 motor vehicle thefts reported in 2023, 241 are electric scooters, bikes, skateboards, or golf carts.

How would you make this MUCH better, ie 'plus' it? In your answer, discuss two aspects: the display of the data (what's better than tables?), and, a modern AGENTIC AI chatbot interface (for querying the data). Be sure to put in a sample question someone would ask the chatbot.

A. Crime ALWAYS involves a location, so data like the above is best visualized on a map (eg. with different symbols denoting the various kinds of crime).

+1 Proposes at least one concrete better-than-table viz (**map**, heatmap, dashboard, sparklines, stacked bars, etc.) with a reason.

+1 Describes an agentic-AI chatbot layer over the data such as text-to-SQL, tool-calling, grounded answers, conversational state, or actions.

+1 Provides a concrete sample user question that the chatbot would answer.

Eg. 'what is the most common kind of crime', 'where does most crime occur (regardless of type)', etc.

Q9 — 3 points

Q9. "MapReduce" is, as you know, 'really speaking', Map-Combine-Shuffle-Reduce
- a loving division of labor among BFFs ☺

- A. how exactly do they divide labor? Explain briefly, each of the 4 steps.
- B. where does speedup happen?
- C. which step is optional?

A.

+1 Part A: correctly characterizes all 4 steps (Map emits k-v pairs; Combine local mini-reduce; Shuffle groups by key across the cluster; Reduce aggregates per key)

+1 Part B: identifies parallelism in **Map** as the source of speedup (since that is where parallel processing happens)

+1 Part C: identifies Combine as the optional step.

Q10 — 3 points

Q10. Two very different kinds of databases (ie based on different data models)
both employ a hierarchical index to speed up lookups.

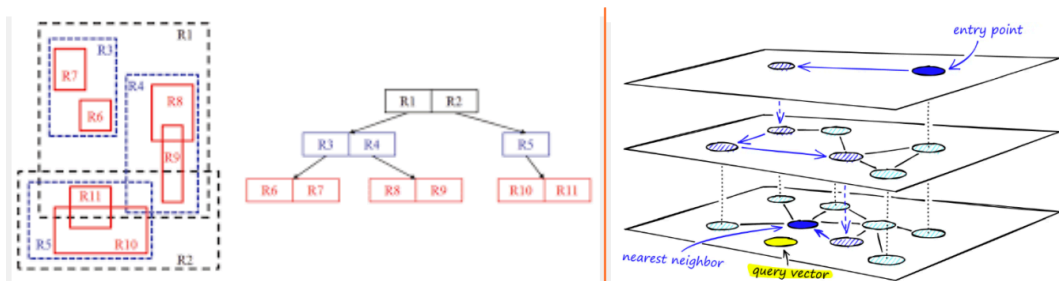
A. which two are they?

B. how does it work? Explain briefly, using a figure or two.

A.

+1 Part A: correctly names **spatial** and **vector** DBs.

+2 Part B: 1 point for correct explanation for each of the database - spatial DBs use R-trees, vector DBs use a similar hierarchical structure to index vectors (eg HNSW).



We can accept any answer that contains the above even in approximate form, eg. the vector DB indexing figure doesn't need to be quite accurate.

Q11 — 3 points

Q11. We get a lot of mileage out of using iteration (looping), in ML.

Discuss briefly, three different DM/ML algorithms that rely on iteration. You don't need to discuss the entire algorithms, just how/where iteration plays a part.

A. **NN, k-means clustering, linear regression**... all use iteration (side note: linear regression for small datasets doesn't need iteration).

+1 x 3: 1 point for each correctly identified iterative DM/ML algorithm and inpoints WHERE the loop is (e.g., gradient descents update rule). Give partial points for correct algo but no explanation

Q12 — 3 points

Q12.

What specifically makes 'supervised' ML possible? Discuss. Note - the question is NOT asking you to define supervised ML.

A.

+1 Names **labels / labeled data / ground truth / (x, y) pairs** as the central enabler.

+2 Explains WHY labels enable learning — they make a loss function / error signal / gradient possible (i.e., the model has something to optimize against). IOW the known labels help quantify the loss during each training epoch's forward pass, which in turns helps gradually reduce it via backprop.

We can accept any answer that explains the above even using loose terminology (as long as it captures what the labels are for).

Q13 — 3 points

Q13.

With regards to k-v databases (eg Redis) and document databases (eg Mongo)...

A. what is similar?

B. what is different?

C. how would we make graph databases be similar to these?

A.

+1 Part A: both are comprised of explicit **JSON** structures (keys, values)

+1 Part B: gives 1+ valid differences - that in k-v DBs, values are 'simple' and 'small'; in document DBs they can be large and arbitrarily complex (ie deeply nested)

+1 Part C: by expressing a **graph as JSON** :)

Q14 — 3 points

Q14.

Describe the origins of modern DB connectivity... there was a pre-WWW period, now we are in the post-WWW one. Briefly describe the technologies involved ("then" and "now").

A.

+1 Pre-WWW: Microsoft's alphabet soup of ODBC, DAO, JET, RDO, and, JDBC

+1 Post-WWW: identifies at least one modern mechanism: REST, GraphQL, MCP, microservices...

+1 provides correct explanation about the idea behind the mentioned mechanism - we can accept any answer that explains even briefly the underlying architecture (ie how the above work)

Q15 — 3 points

Q15.

DATA => DAgtA. "Discuss" ☺

In other words, how might the worlds of databases and agentic AI collide? Briefly explain 3 aspects. Hint - lectures, HWs... :)

A. Agentic AI can be used to create schemas (eg ER diagrams), create knowledge graphs (a form of graph DB), query DBs, to optimize them, to even create them on demand (eg TiDB or Falkor)...

+1 x 3: 1 point for each aspect identified correctly - 'loose' but valid answers are ok!